Timed-Automata Verification of Payload-Link Supervision, Watchdog Gating, and A/B Boot Policy in STM32 Flight Software

Anonymous Author 1
Anonymous Department
Anonymous Faculty
Anonymous University
anonymous1@anonymous.edu

Anonymous Author 2
Anonymous Department
Anonymous Faculty
Anonymous University
anonymous2@anonymous.edu

Anonymous Author 3
Anonymous Department
Anonymous Faculty
Anonymous University
anonymous3@anonymous.edu

Anonymous Author 4
Anonymous Department
Anonymous Faculty
Anonymous University
anonymous4@anonymous.edu

Anonymous Author 5
Anonymous Department
Anonymous Faculty
Anonymous University
anonymous5@anonymous.edu

Anonymous Author 6
Anonymous Department
Anonymous Faculty
Anonymous University
anonymous6@anonymous.edu

***Abstract*—Small-satellite flight software relies on interacting safety mechanisms – poll/deadline supervision of a payload link, a task-health-gated hardware watchdog and a CRC-validated A/B bootloader – that are critical for recovery but hard to verify as a whole. Bench evaluation of such mechanisms reports host-observed latencies and cannot establish worst-case bounds or reach states that a test campaign never exercised, and the closest formal-verification work reviewed here addresses protocols and kernels; the composed mechanisms of one prototype, checked against its own hardware traces, are not reported in the cited papers. This work verifies an abstracted design model of the three interacting mechanisms of an STM32 flight-software prototype as UPPAAL timed automata, with constants copied from the C++ source, against safety, bounded-response and liveness properties in TCTL. Across fifteen model configurations and 254 formulas every verdict matches a pre-declared oracle: after confirmed fault onset the first timeout occurs within 605 ms and OFFLINE within 1609 ms, recovery completes within 506 ms, a stalled critical task is detected within 952 ms and forces a watchdog reset, and the boot policy never launches an unvalidated image; each bound is tight in the model. Model checking yields three design consequences and exposes one defect, confirmed and fixed in source: a bypassed unconfirmed-boot limit under which a Pending fallback image receives one attempt instead of four. We then evaluate the verified models against frozen, hash-verified hardware traces from 90 fault-injection trials and further datasets, re-anchored at the confirmation events where the model's clocks start, as non-refuting consistency evidence: 179 of the 180 recorded intervals lie inside the bounds, and the exception, 17 ms over, is reported as unresolved. The guarantees concern the abstracted design under stated jitter and channel assumptions; the work does not verify the deployed C++ implementation, the RTOS or the HAL.**

***Index Terms*—model checking, timed automata, UPPAAL, embedded flight software, watchdog, bootloader, payload link**

I. Introduction

Resource-constrained flight computers for CubeSat-class missions rely on a small set of deterministic safety mechanisms: a polling supervisor that detects a silent, corrupt, or late payload; a task-health registry that gates refresh of an independent hardware watchdog; and a bootloader that validates an image before jumping to it and otherwise falls back to a second slot or a safe beacon loop. They are simple to state and easy to bench-test, yet a campaign only shows the paths it drove, and host timestamps cannot separate controller timing from serial and OS overhead. An earlier, unpublished hardware-in-the-loop (HIL) campaign on the same prototype, whose frozen traces we reuse in Section IV-C, observed the predefined markers in 30/30 trials per fault mode and could not claim controller-internal timing for that reason.

This paper asks what the design itself guarantees. We formalize the three mechanisms of an STM32 flight-software prototype (name withheld for review; firmware not yet released) as UPPAAL timed automata [1], [2], [3] with constants copied from the source, verify properties with `verifyta`, and use the frozen HIL traces not as the result but as offline consistency evidence for the models.

The contributions are bounded as follows. (1) Timed-automata models and verified properties for the payload-link supervisor, the task-health watchdog gate and the A/B boot policy of one prototype: worst-case detection, OFFLINE, recovery and stall-to-reset bounds, each shown attained, the delay-acceptance boundary, and ordering, safety, boundedness and progress properties (Table III). (2) One defect found by model checking and three design consequences made precise by the models, all confirmed in source (Section IV-B). (3) Model-to-hardware consistency evidence from offline replay of frozen, hash-verified traces against the model's alphabet and bounds, under an anchor aligned with the model's clocks and an uncertainty quantified from the evidence itself. The replay can refute agreement; it does not establish conformance in the

testing sense. **Assurance limits.** We do not claim verification of the C++ implementation, correctness of the RTOS or vendor HAL, robustness to environmental or radiation effects, or any reliability estimate (Section V).

II. Related Work

Timed automata [1] and UPPAAL [2], [3] are the standard formalism and tool; we position the paper along five layers of the verification stack, then against work on the same mechanisms.

RTOS semantics. Singh et al. apply UPPAAL model checking to conformance testing of an ARINC 653 RTOS [4]; Lin and Wang model the FreeRTOS Cortex-M4 port, find errors by model checking and reproduce the assertion failures on an STM32F429 board [5]. Both verify the kernel; we verify application-level supervision above an abstracted kernel, so scheduler-originated faults lie outside our results (Section V). UPPAAL has also been applied to satellite communication software [6]; link-supervision bounds are not reported in the cited paper.

Compositional scaling. State explosion is a recognized limit of real-time model checking, met with compositionality and abstraction [7]. Our state spaces stay small (at most 12 547 states) because the three mechanisms are verified as three networks whose coupling (the link task is a monitored task; a stall leaves the retained fault the boot policy reads) enters each network as an input, so a further task or link enlarges one network without multiplying it by the others.

Property engineering. UPPAAL checks only non-nested reachability, safety and liveness, so richer real-time requirements need observer automata, which pattern catalogs now generate [8]. Our properties were written by hand from the research questions, encoding bounded response over the clocks of the scenario's driver template (Section V), and were not cross-checked against the catalog.

Model-to-code assurance. Integrated formal verification of safety-critical software [9], verified code generation from timed automata with runtime monitors [10], and UPPAAL-TRON, which drives an implementation online and yields a conformance verdict on it [11], all close the model–code gap more strongly than we do. We replay recorded, hash-verified traces against the model's alphabet and bounds without driving the implementation, so an accepted trace shows only that the run did not contradict the model. The choice is deliberate: the only free interface is the debug UART that already carries the console markers, and exposing internal events would perturb the poll timing under test.

Hybrid V&V. Pontes et al. combine model checking with the CoFI conformance-and-fault-injection methodology for space embedded software [12]; fault injection [13], [14] is the empirical counterpart. Formal verification is one layer of that case here, and the reused HIL campaign serves only as data.

The same mechanisms. Pan et al. verify CAN arbitration and fault confinement in UPPAAL [15] and Johansson et al. a heartbeat-based failure detector [16]; neither reports a poll/deadline supervisor with a consecutive-timeout threshold. Ahmed and Safar verify an AUTOSAR Watchdog Manager by symbolic execution [17]; Cutler and Beningo report a nanosatellite watchdog with separate clock and power domains that recovers restart-curable faults in flight [18]; Asch et al. frame software-defined watchdogs as eager deadlines for delayed or missing signals [19]. Those designs emphasize recovery, whereas the gate verified here is fail-safe (F2). Boot code has been verified for memory safety [20] and secure-boot properties [21]; slot selection, unconfirmed-boot limits and fallback across resets are not reported in the cited papers.

III. Methodology

A. System under study

The controller is a NUCLEO-G431RB (STM32G431RB, 128 KiB flash, 32 KiB RAM) running a FreeRTOS application with static allocation and no heap. Every constant below is quoted from the source and traced to its line in the artifact; Table II shows representative rows, including the source anchor of finding F4.

Payload-link supervisor. A 32-byte CRC-32 framed poll is sent every $P = 500\,\text{ms}$. A valid response must arrive within $D = 100\,\text{ms}$ of the send instant; the deadline and the next poll are both anchored at that instant. On a timeout the consecutive-timeout counter increments and the link enters OFFLINE when it reaches $N = 3$. A valid response resets the counter, sets ONLINE, and counts a recovery if the link was OFFLINE. A CRC-corrupted frame increments a counter but leaves the outstanding poll pending; a valid frame that arrives when no poll is pending is a sequence reject. The task services the controller once per $L = 1\,\text{ms}$ loop, receiving frames before evaluating the deadline.

Watchdog gate. Two critical tasks check in periodically: the heartbeat every 500 ms with a 1500 ms deadline, and the payload-link task every 100 ms with an 800 ms deadline. A watchdog task wakes every 50 ms and evaluates the registry once 100 ms have elapsed since the previous evaluation, expiring an entry whose last check-in is older than its deadline, and the first expiry records a watchdog-timeout and a safe-fail event in retained memory. The independent watchdog (IWDG, prescaler 16, reload 3999, LSI $\approx$32 kHz, nominal 2.0 s) is refreshed only while all critical entries are healthy. On this target the safe-fail callback does not reset the processor; the IWDG does.

A/B boot policy. On reset the bootloader recovers a CRC-protected retained fault log and boot metadata, applies a confirmed-boot flag if set, refuses to boot if the active slot is Pending with a boot count above 3 or the fault log holds a check-in failure or watchdog timeout, validates the active slot's state, vector table and CRC-32, falls back to the other slot, and otherwise records a safe-fail and enters an LED beacon loop.

B. Payload-link model

The network has three templates. *Controller* carries clocks for time since the last poll and since the outstanding poll, and its transitions mirror `preparePoll`, `acceptResponse` and

TABLE I

SCOPE PER MECHANISM: WHAT THE MODEL ABSTRACTS AND WHAT IMPLEMENTATION EVIDENCE EXISTS.

Mechanism	Abstracted in the model	Implementation evidence
Payload link (Q)	RTOS and ISR latency as jitter J=2 ms; frame time $T_{tx} \in [3, 4]$ ms; lossless channel	offline replay: 90 trials, 3 nominal windows, delay sweep, 2 F411 pairs
Watchdog gate (W)	task loop and poll as $50{+}100{+}J$; IWDG period as the LSI bracket $[1882, 2170]$ ms	native unit tests only; no stall in the frozen traces
Boot policy (B)	untimed; flash and application as environment choices	native regression test for F4; no trace evidence

TABLE II

REPRESENTATIVE MODEL-TO-SOURCE ROWS (FULL TRACE IN THE ARTIFACT); LINE NUMBERS ARE FOR THE ANALYZED REVISION, BEFORE THE F4 FIX (SECTION IV-B).

Model element	Source
P, D, N = 500, 100, 3	`PayloadLinkController.hpp:32-34`
Idle→Await; next poll anchored at the send instant	`preparePoll() :53-73 (:69)`
Await→Await on bad CRC: poll stays pending	`acceptResponse() :84-90`
Receive before deadline evaluation	`PayloadLinkTask.hpp:45-47`
Supervisor expiry is strict-greater	`TaskHealth.hpp:96-104`
Refresh only while all critical tasks healthy	`Watchdog.hpp:61-65`
Limit 3, evaluated on the active slot (F4)	`BootPolicy.hpp:16, :126-131`
Fallback activates the other slot, no re-check (F4)	`BootPolicy.hpp:86-92`

`service`. *Payload* answers a poll after a serialization interval $[3, 4]$ ms, preceded by a configured delay d in DELAYED mode, or not at all in SILENT mode, or with a corrupted CRC in BAD_CRC mode. *Host* reproduces the frozen trial protocol: once the link is ONLINE it injects one fault mode, holds it for the campaign's 4 s exposure, restores NORMAL, and observes for 3 s; its transitions are the confirmation events at which the comparison anchors start (Section III-F). A response arriving with no poll pending is a sequence reject.

Abstractions: RTOS wakeup and ISR latency are bounded by a 2 ms jitter constant (assumed, not measured); the channel is lossless except for the injected corruption; $d < P$ so responses never overlap the next poll. Which results rest on the jitter abstraction is explicit: Q1, Q7, Q8, W1, W2 and every boot property are stated without J, and each bounded-response and boundary bound is a closed form in J (Table III), so a larger measured jitter moves a bound without invalidating the proof. No value of J covers a scheduler-induced delay beyond it, such as blocking or priority inversion; Section V states why the code excludes it.

C. Watchdog-gate model

Two *Task* instances check in with their periods and may be stalled by an *Env* injector; a *Supervisor* evaluates every 100 ms with the source's strict-greater expiry test and refreshes the *Iwdg* only when both are healthy; the IWDG resets when its clock reaches a value in $[1882, 2170]$ ms, derived from the datasheet LSI range of 29.5–34 kHz over the full supply and temperature range [22]. Scenario 0 never stalls; Scenario 1 stalls one task after warm-up.

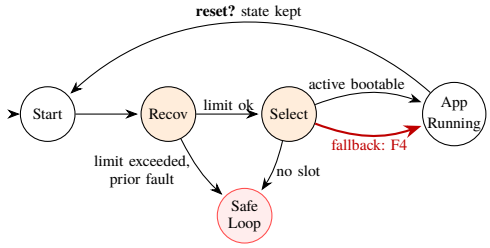


Fig. 1. The A/B boot policy; committed locations shaded. The check that bounds unconfirmed boots runs once, on the *active* slot, on the edge leaving Recov. The red edge is the fallback: it activates the other slot and launches it without re-evaluating that check, so the launch inherits the accumulated boot count. That is finding F4.

D. Boot-policy model

An untimed network (Fig. 1): *Boot* runs the policy over the retained variables (active slot, slot states, boot count, confirmed flag, retained critical fault, metadata corruption); *App* may stall, setting the retained fault and resetting, reset cleanly, or, in a design variant, set the confirmed flag; *Flash* corrupts an image, stages a Pending image in slot B, or cold-starts the board, scrubbing retained SRAM. The shipped variant gives the application no confirm capability because no code under the application tree writes boot metadata.

E. Properties

Table III lists the properties referenced in this paper; the complete set of 19, 11 and 14 formulas for the three models is in the artifact.

F. Consistency-evidence method

The frozen traces were acquired with a confirmation-gated protocol: a host injects a payload fault mode over the payload board's debug UART, counts the trial only after the payload confirms the mode, observes the controller console for 4 s, restores NORMAL, counts recovery only after the payload confirms restoration, and observes for 3 s; both serial channels are logged byte-exact with host timestamps and hash-inventoried.

The campaign reports each interval from the command send on a monotonic clock, but the model's clocks start at the confirmed mode change and the confirmed restoration, so the command anchor is structurally earlier and overstates the model-comparable interval. We therefore recompute every interval from the retained wall-clock stamp of the confirming serial event to that of the observed marker. Three uncertainties remain. Every command-anchored interval carries both a wall-clock and a monotonic stamp, and their disagreement bounds the wall-clock granularity at 15.4 ms, with a 4.4 ms median over the 180 intervals. Cross-channel read latency between the payload and controller serial channels is not separately measured. The clock-rate difference is systematic and the evidence measures it: the model's constants are in MCU time and the intervals in host time, and each nominal window carries the controller's poll counter beside host timestamps,

TABLE III

PROPERTIES (TCTL). P=500, D=100, L=1, N=3, J=2, $T_{tx} \in [3, 4]$ MS. BOUNDS ARE INCLUSIVE; EACH HAS A TIGHTNESS TWIN USING $\geq$ THAT IS EXPECTED TO FAIL.

Id	Kind	Statement
Q1	safety	nominal, lossless, link up $\Rightarrow$ never OFFLINE
Q2	bounded	fault confirmed for $> P{+}J{+}D{+}L{+}J = 605 \Rightarrow$ a timeout was counted
Q3	bounded	fault confirmed for $> N(P{+}J){+}D{+}L{+}J = 1609 \Rightarrow$ OFFLINE
Q4	bounded	NORMAL confirmed for $> P{+}J{+}T_{tx}^{\max} = 506 \Rightarrow$ ONLINE
Q5	liveness	Restored $\rightsquigarrow$ ONLINE
Q7/Q8	order	BAD_CRC: first timeout preceded by a CRC reject; no timeout yet $\Rightarrow$ still ONLINE
Q9	boundary	DELAYED: $d{+}T_{tx}^{\max}{<}D \Rightarrow$ no timeout; $d{+}T_{tx}^{\min}{>}D{+}L{+}J \Rightarrow$ late frames sequence-rejected; otherwise both outcomes reachable
Q0/Q11	progress	no deadlock; Awaiting $\rightsquigarrow$ Idle and Idle $\rightsquigarrow$ Awaiting
W1(a,b)	safety	no stall $\Rightarrow$ never reset (a), never critical failure (b)
W2	safety	no IWDG refresh after a critical failure is recorded
W3	bounded	stall detected within deadline $+100{+}50{+}J$: 952 (link, deadline 800), 1652 (heartbeat, 1500)
W5/W6	liveness	stall $\rightsquigarrow$ IWDG reset, within detection bound + IWDG max (2170)
B1/B2	safety	never launch an unvalidated image or a non-bootable slot state
B3	bounded	Pending active slot launched at most 4 times
B4/B5	safety/reach	no valid image $\Rightarrow$ safe loop; safe loop reachable
B6	safety	retained critical fault $\Rightarrow$ application never relaunched before cold start
B7(,r)	finding	shipped app: Pending never becomes Confirmed; reachable only in the design variant (r)
B8(b,c)	finding	unconfirmed-boot limit never bypassed (fails); bypass only via fallback (b); next warm boot refused (c)
B9	progress	Start $\rightsquigarrow$ AppRunning $\vee$ SafeLoop

and over 1200 consecutive polls the mean period is 498.83 ms against a nominal 500 ms, the two 65 s windows agreeing to within 0.03 %, so the MCU base runs 0.235 % fast and host intervals are shorter by that factor, 1.4 ms at the first-timeout bound; the mean falling *below* nominal also shows that the per-poll overhead bounded by the 2 ms jitter constant is not consumed on average. We therefore declare a falsification floor of 50 ms, more than three times the demonstrated clock disagreement: the replay can refute a bound violation larger than the floor, and reports any smaller excess as unresolved instead of absorbing it.

IV. RESULTS

A. Verified properties and bounds

All configurations were verified with UPPAAL 5.0.0 (`verifyta`, rev. 714BA9DB) in one batch of 254 formulas whose outputs, instantiated models and SHA-256 inventory are retained in the artifact (Table IV). Every verdict equals a pre-declared oracle stating, per configuration, which formulas must hold and which must fail. The largest state space is 12 547 symbolic states (boot policy, design variant); no query took more than 0.5 s of CPU on an Intel Core Ultra 7 155H.

Every model satisfies `A[] not deadlock`, though in the boot network that is a modeling sanity condition, since its environment templates always offer an enabled edge; progress is therefore stated explicitly: every outstanding poll is resolved

TABLE IV

VERIFICATION BATCH: 15 CONFIGURATIONS, 254 FORMULAS, ALL VERDICTS EQUAL TO THE ORACLE (S SATISFIED, N NOT SATISFIED AS EXPECTED; Q9C IS TWO REACHABILITY FORMULAS; TWO ROWS SWEEP J AT THE BOUNDARY DELAY).

Model	Configuration	States	Verdicts (N = expected fail)
payload	SILENT	68	14 S, 5 N: Q2T, Q3T, Q4T, Q9c
payload	BAD_CRC	96	14 S, 5 N: Q2T, Q3T, Q4T, Q9c
payload	DELAYED 50, 90	68	17 S, 2 N: Q9c
payload	DELAYED 100	652	18 S, 1 N: Q4T; Q9c both reachable
payload	DELAYED 110, 250	98	14 S, 5 N: Q2T, Q3T, Q4T, Q9c
payload	DELAYED 100, $J{\in}\{0,1\}$	98, 113	14 S, 5 N: as DELAYED 110
payload	DELAYED 100, $J{=}5$	586	18 S, 1 N: as DELAYED 100
watchdog	no stall	314	11 S
watchdog	one stall	807	8 S, 3 N: W1a, W1b, W3T
boot	shipped app	3 035	12 S, 2 N: B7r, B8
boot	design variant	12 547	11 S, 3 N: B7, B8, B8c
boot	F4 fix applied	1 988	13 S, 1 N: B7r

and the controller polls again (Q11), a stall leads to a reset (W5), and every reset ends in a launch or the safe loop (B9). The bounds are inclusive and tight: as a strict inequality each formula holds and with $\geq$ each fails, so the event can occur exactly at the bound. They exceed a naive hand derivation by one jitter term per poll period, because the controller anchors the next poll at the actual send instant so wake-up jitter accumulates; the model caught that omission in our own first estimate.

Sensitivity to the jitter bound. The first-timeout bound is $601 + 2J$ ms; the delay-acceptance boundary is more informative: the accepted region $d + T_{tx}^{\max} < D$ does not involve J, so its edge stays at 95 ms, whereas the timed-out region $d + T_{tx}^{\min} > D{+}L{+}J$ moves with J, leaving an undetermined band of 96 to $98{+}J$ ms. Re-verifying at $J \in \{0, 1, 5\}$ ms confirms this: for $J \leq 1$ ms the model decides $d = 100$ ms as a timeout, as the hardware did; from $J = 2$ ms it is undecidable. The band is therefore a consequence of a conservative jitter bound rather than a limit of the model, and measuring J would remove it.

B. Design findings

Model checking produced one defect (F4) and made three design consequences precise (F1–F3). For F1 and F2 the models quantify behavior that source inspection had already identified; only F4 was discovered by the checker.

F1: the boot-confirmation path is unreachable in flight. B7 holds in the shipped configuration and fails in the design variant. Inspection established the source fact first – no file under the application tree references the boot metadata or the confirmed flag – and it was then encoded as the shipped configuration's zero confirm capability, so the model contributes the consequence, not the discovery: a slot provisioned as Pending can leave that state only through the bootloader's attempt limit (B3, B8), never through confirmation. The prototype is safe

because slot A is provisioned as Confirmed at build time, not because confirmation is exercised.

F2: critical stalls are fail-safe, not fail-operational. W2 and W6 show that once a critical task expires the IWDG is never refreshed again and a reset follows within one IWDG period. B6 then shows the bootloader, seeing the retained watchdog-timeout event, enters the safe loop instead of relaunching until a cold start – a composed behavior the component tests did not make visible.

F3: the effective delay boundary is below the nominal deadline. Q9 proves, in the model, acceptance whenever $d + T_{tx}^{\max} < D$ ($d \leq 95$ ms) and a timeout with sequence-rejected late frames whenever $d + T_{tx}^{\min} > D + L + J$ ($d \geq 101$ ms); at $d = 100$ ms both outcomes are reachable (Q9c), because the earliest response and the latest deadline check coincide at 103 ms under the 2 ms jitter assumption. The frozen sweep (Section IV-C) observed 90 ms accepted and 100 ms rejected in every valid trial on three board pairs, consistent with the model but not proven by it.

F4: the fallback path bypasses the unconfirmed-boot limit. B8 fails in both boot configurations and B8b shows the violation exists only on the fallback path. The limit check reads the state and boot count of the *active* slot; when that slot fails validation and the other slot is Pending (a staged, unconfirmed update), the fallback launch inherits the boot count accumulated while the confirmed slot was running. B8c then shows that the next warm boot is refused. The counts make the defect exact. Every launch of either slot increments the count, only confirmation resets it, and the limit refuses a Pending active slot whose count exceeds 3, so a freshly staged Pending slot is launched at counts 0–3 and refused at 4: four launches (B3). After k warm resets of a Confirmed slot the count is k and unchecked; when that slot then fails validation, the fallback launches the Pending slot at count k without a check, so it is refused after $4 - k$ launches – after one once $k \geq 3$, the case of the regression test (launch at 3, count 4, next boot refused). A Pending fallback image therefore gets one attempt instead of four. The fix – reset the attempt budget when the active slot changes – was re-verified: B8 holds with every other boot property unchanged and B8c becomes vacuous; a native regression test reproduces the inherited count and fails without the fix.

C. Consistency with frozen hardware traces

The replay accepts every frozen dataset. In all 90 primary trials the predefined detector marker is the model's first observable event for that mode, and no trial recorded a controller restart marker. Both 65 s nominal windows and the separate 605 s observation show zero counter deltas, as Q1 requires. In the delay sweep the 6 rows at 50 and 90 ms were accepted and the 9 rows at 110, 150 and 250 ms timed out, as Q9a and Q9b predict, while the 3 rows at 100 ms fall inside the undetermined band; the two F411 pairs add 18 consistent rows at 90 and 110 ms and in the nominal condition, and 6 rows at 100 ms, likewise undetermined. The 2 BAD_CRC mechanism traces show the CRC reject preceding the first timeout, as Q7 requires.

TABLE V
HOST-OBSERVED INTERVALS OF THE 90 PRIMARY TRIALS AGAINST THE VERIFIED BOUNDS (MS; $n = 30$ PER MODE); THE COMMAND ANCHOR IS THE CAMPAIGN'S INTERVAL FROM THE COMMAND, THE CONFIRMATION ANCHOR STARTS WHERE THE MODEL'S CLOCK DOES AND GIVES THE WITHIN-BOUND COUNTS.

			Cmd.	Confirmation		Within
Mode	Interval	Bound	max	median	max	bound
SILENT	detection	605	578	346	550	30/30
BAD_CRC	detection	605	531	297	530	30/30
DELAYED 250	detection	605	610	385	553	30/30
SILENT	recovery	506	516	243	456	30/30
BAD_CRC	recovery	506	531	322	523	29/30
DELAYED 250	recovery	506	531	222	297	30/30

Table V compares the recorded intervals with the verified bounds under both anchors. Under the confirmation anchor 179 of the 180 intervals lie within the bounds. The exception is one BAD_CRC recovery interval of 523 ms against the 506 ms bound, an excess of 17 ms (18 ms after the clock-rate correction), larger than the 15.4 ms clock disagreement the evidence demonstrates, so timestamp granularity does not fully account for it; the remainder lies within the unmeasured cross-channel read latency, and the interval is reported as unresolved rather than as consistent or violating. No interval exceeds a bound by more than the 50 ms falsification floor, so the replay finds no refutation of the model's bounds at the resolution the recorded evidence supports.

At the campaign's own anchor four of the six maxima exceed the bounds by 5 to 25 ms, so the same traces would appear to violate bounds they in fact respect.

V. Discussion

A. Threats to validity and limitations

The models are hand-derived. Deadlock freedom and progress are properties of the models, not of the firmware's task interactions; task-level blocking is excluded by construction, since the flight code uses no semaphores, queues, mutexes, event groups or task notifications and blocks only in delays, non-blocking critical sections and UART calls with a 10 ms timeout – verified by source inspection, not by the models. The transition-to-source trace and the offline replay narrow the gap but do not prove refinement; a code-level check of the header-only controller with a bounded model checker is future work. The jitter bound's worst case is not measured; serialization is derived, not measured; the channel is assumed lossless apart from the injected corruption; and the IWDG bracket comes from the datasheet LSI specification [22]. The bounds are parametric rather than platform-specific: Table III states each as a closed form in P, D, L, N, J and T_{tx}, of which only J and T_{tx} depend on the MCU and RTOS. Another target therefore needs those two constants re-measured and the same queries re-run, not new models. The consistency data come from one primary board pair and two further pairs of a different MCU family under one fixed configuration, and say nothing about device populations. Nothing here addresses

environmental, electrical, or radiation-induced faults, and nothing constitutes qualification evidence. The empirical support is uneven (Table I): only the link bounds are replayed against traces; the watchdog and boot results rest on model checking, source inspection and native tests, and injecting stalls and boot faults is future work.

Property expressiveness. UPPAAL checks a restricted TCTL fragment – invariants, reachability and leads-to, without nesting. Of the 44 formulas, 34 are invariants, 4 reachability and 6 leads-to. Bounded response is therefore an invariant over a clock of the driver template that starts at the confirmation event, so the driver plays the observer role a pattern catalog would generate [8]; ordering properties use Boolean flags set on the controller's transitions. No alternative encoding was cross-checked, and a mis-stated phase predicate would pass both tightness twins. As inequalities over console-observable events the bounds could also run as monitors on the debug UART, as model-paired runtime monitoring does [23], [10]; we have not done so.

B. Reproducibility

The artifact (models and queries, batch script with retained `verifyta` outputs and SHA-256 inventory, verdict oracle and checker, line-numbered model-to-source trace, trace replay with unit tests and JSON report, frozen HIL datasets with inventories) is released on acceptance; the firmware is not, so the trace is pinned to a tagged internal revision and source excerpts can be supplied through the chairs.

VI. Conclusion

Model checking the composed supervisor, watchdog gate and boot policy of an STM32 flight-software prototype yields worst-case bounds a bench cannot provide and exposes one defect in shipped safety logic with three design consequences. All 254 verdicts across fifteen configurations equal a pre-declared oracle, the defect is fixed and the fix re-verified, and replaying frozen hardware traces refutes none of the bounds at the resolution the recorded evidence supports. It shows non-refuting trace consistency with hardware behavior and does not verify the deployed C++ implementation, the RTOS or the HAL.

Acknowledgment

The authors thank the institute that provided hardware and technical consultation for this work; its name is withheld for double-anonymous review. The authors used Claude and ChatGPT only to translate the original Thai text into English and to refine grammar, not for the research, models, verification or analysis. All content, data and conclusions are the authors' own; they reviewed and approved the final text and take full responsibility for its accuracy and integrity.

References

[1] R. Alur and D. L. Dill, "A theory of timed automata," *Theor. Comput. Sci.*, vol. 126, no. 2, pp. 183–235, 1994.

[2] K. G. Larsen, P. Pettersson, and W. Yi, "Uppaal in a nutshell," *Int. J. Softw. Tools Technol. Transf.*, vol. 1, no. 1–2, pp. 134–152, 1997.

[3] G. Behrmann, A. David, and K. G. Larsen, "A tutorial on Uppaal," in *Formal Methods for the Design of Real-Time Syst.*, ser. LNCS. Springer, 2004, vol. 3185, pp. 200–236.

[4] A. Singh, M. D'Souza, and A. Ebrahim, "Conformance testing of ARINC 653 compliance for a safety critical RTOS using UPPAAL model checker," in *Proc. 36th Annu. ACM Symp. Applied Computing (SAC)*, 2021, pp. 1807–1814.

[5] C.-K. Lin and B.-Y. Wang, "Formal analysis of FreeRTOS scheduler on ARM Cortex-M4 cores," in *Formal Methods and Software Engineering (ICFEM 2024)*, ser. LNCS. Springer, 2024, vol. 15394, pp. 199–215.

[6] W. A. F. Alencar and E. Villani, "Model checking applied to embedded software of university satellite," *J. Control Autom. Electr. Syst.*, vol. 25, no. 1, pp. 126–136, 2014.

[7] H. E. Jensen, K. G. Larsen, and A. Skou, "Scaling up Uppaal: Automatic verification of real-time systems using compositionality and abstraction," in *Formal Techniques in Real-Time and Fault-Tolerant Systems (FTRTFT 2000)*, ser. LNCS. Springer, 2000, vol. 1926, pp. 19–30.

[8] T. Vogel, M. Carwehl, G. N. Rodrigues, and L. Grunske, "A property specification pattern catalog for real-time system verification with UPPAAL," *Inf. Softw. Technol.*, vol. 154, p. 107100, 2023.

[9] N. Ge, E. Jenn, N. Breton, and Y. Fonteneau, "Integrated formal verification of safety-critical software," *Int. J. Softw. Tools Technol. Transf.*, vol. 20, no. 4, pp. 423–440, 2018.

[10] S. Cho, I. Kang, and J. H. Kim, "From timed automata to Go: Formally verified code generation and runtime monitoring for cyber-physical systems," *IEEE Access*, vol. 13, pp. 161 729–161 749, 2025.

[11] K. G. Larsen, M. Mikucionis, B. Nielsen, and A. Skou, "Testing real-time embedded software using UPPAAL-TRON: An industrial case study," in *Proc. 5th ACM Int. Conf. Embedded Software (EMSOFT)*, 2005, pp. 299–306.

[12] R. P. Pontes, P. C. Véras, A. M. Ambrosio, and E. Villani, "Contributions of model checking and CoFI methodology to the development of space embedded software," *Empir. Softw. Eng.*, vol. 19, no. 1, pp. 39–68, 2014.

[13] J. Arlat, M. Aguera, L. Amat, Y. Crouzet, J.-C. Fabre, J.-C. Laprie, E. Martins, and D. Powell, "Fault injection for dependability validation: A methodology and some applications," *IEEE Trans. Softw. Eng.*, vol. 16, no. 2, pp. 166–182, 1990.

[14] C. L. G. Batista, A. C. Weller, E. Martins, and M. de Fátima Mattiello-Francisco, "Towards increasing nanosatellite subsystem robustness," *Acta Astronaut.*, vol. 156, pp. 187–196, 2019.

[15] C. Pan, J. Guo, L. Zhu, J. Shi, H. Zhu, and X. Zhou, "Modeling and verification of CAN bus with application layer using UPPAAL," *Electronic Notes in Theor. Comput. Sci.*, vol. 309, pp. 31–49, 2014.

[16] B. Johansson, M. Rågberger, A. V. Papadopoulos, and T. Nolte, "Heartbeat bully: Failure detection and redundancy role selection for network-centric controller," in *Proc. 46th Annu. Conf. IEEE Industrial Electronics Society (IECON)*, 2020, pp. 2126–2133.

[17] M. Ahmed and M. Safar, "Formal verification of AUTOSAR watchdog manager module using symbolic execution," in *Proc. 30th Int. Conf. Microelectronics (ICM)*, 2018, pp. 240–243.

[18] J. W. Cutler and J. Beningo, "Watchdog-based fault recovery on nanosatellites," *J. Aerosp. Inf. Syst.*, vol. 19, no. 8, pp. 522–529, 2022.

[19] B. Asch, E. Jellum, M. Lohstroh, and E. A. Lee, "Software-defined watchdog timers for cyber-physical systems," *IEEE Embedded Syst. Lett.*, vol. 17, no. 2, pp. 115–118, 2025.

[20] B. Cook, K. Khazem, D. Kroening, S. Tasiran, M. Tautschnig, and M. R. Tuttle, "Model checking boot code from AWS data centers," *Formal Methods Syst. Des.*, vol. 57, no. 1, pp. 34–52, 2021.

[21] S. Vasudevan, P. Ravi, A. Jati, S. Bhasin, and A. Chattopadhyay, "Formal verification of secure boot process," in *Proc. Design, Automation & Test in Europe Conf. (DATE)*, 2024, pp. 1–6.

[22] STMicroelectronics, *STM32G431xB Datasheet – Production Data*, STMicroelectronics, Oct. 2021, DS12589 Rev. 6.

[23] S. Dal Zilio, P.-E. Hladik, F. Ingrand, and A. Mallet, "A formal toolchain for offline and run-time verification of robotic systems," *Robot. Auton. Syst.*, vol. 159, p. 104301, 2023.